SOFTWARE ENGINEERING LAB 3 - Architecture Justification

Component Modelling and Architectural Pattern Selection

Team 08 | Problem Statement #08 - Digital Campus Library Reservation Gateway | Namith Kumar C (PES1UG24AM171), Section C

Architecture Selection

We chose the Microservices Architecture for the Digital Campus Library Reservation Gateway System. The system is split into five independently deployable services - Catalog, Reservation, Circulation, Fine and Notification - behind a single API Gateway, each owning its own database (see the component diagram).

Style Comparison (Step 2)

Style	Strengths for this scenario	Weaknesses for this scenario	Fit
Layered	Simple, clear separation of concerns, easy for a small team.	One deployable unit: search load cannot be scaled alone; one failing layer or node affects every feature (NFR-002).	Medium
Client-Server	Central control, simple deployment, easy data consistency.	Single point of failure and scalability bottleneck; conflicts with 99.9% uptime and 5-minute recovery.	Low
Microservices	Independent scaling, fault isolation, independent deployment, per-service data ownership.	Operational complexity, network latency, eventual consistency between services (mitigated below).	High

Two Reasons for the Choice

1. Availability and fault isolation (NFR-002). The library must be up 99.9% of the semester and recover from a single-node failure within 5 minutes. With microservices, a crash in the Fine or Notification service does not stop members from searching the catalog or placing holds, and each service runs as multiple replicas that an orchestrator restarts automatically. In a layered or client-server design one failing node would take down every feature.

2. Uneven load across features (NFR-001, FR-002). Catalog search must return paginated results for 50,000 titles in under 150 ms and spikes at enrollment and exam time, while fine calculation and hold-expiry timers are light, scheduled work. Separate services let us scale the Catalog Service alone and deploy changes to Fines without touching the search path.

Security Advantage

All traffic enters through the API Gateway, which terminates TLS, validates the JWT and enforces role-based access, so only the Head Librarian role can reach the add / update / deactivate catalog operations (FR-004) and a Student Member can see only their own holds and fines. Services sit on a private network, and database-per-service ownership means a compromised service cannot read another service's data (for example, fine records or members' contact details used for notifications).

Performance Benefit

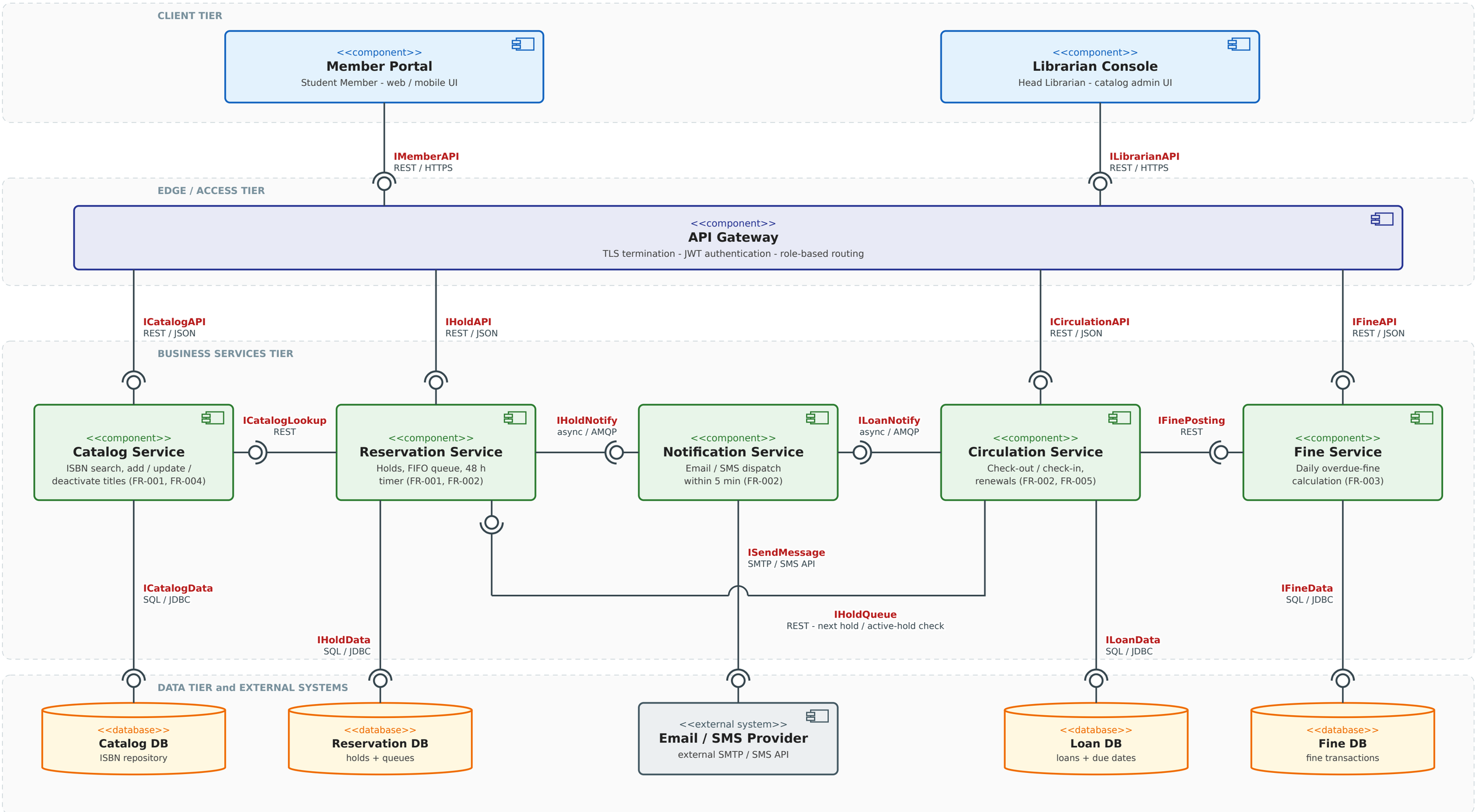
The Catalog Service can be scaled horizontally and given its own read cache and search index, which is how the 150 ms search target is met under peak load without paying for extra capacity elsewhere. Notifications are sent asynchronously over a message queue, so a book check-in returns immediately and the member is still notified within the 5-minute limit in FR-002 even if the email / SMS provider is slow.

Trade-off and Mitigation

Microservices add network calls and eventual consistency (for example, a hold is promoted before the notification is delivered). We accept this: calls between services use short timeouts and retries, and the hold queue is the single source of truth in the Reservation Service, so FIFO order is never affected by a delayed message.

Digital Campus Library Reservation Gateway (DL2KS) - UML Component Diagram

Architectural style: Microservices | SE Lab 3 - Team 08



Notation key

- Provided interface (ball) - service the component offers
- Required interface (socket) - service the component needs

Ball inside socket = assembly connector (socket = consumer, ball = provider).
Solid line = synchronous call (REST / SQL); AMQP = async queue.
Arc on a line = lines cross without connecting.

- Client tier
- Edge tier
- Microservice
- Data store (database-per-service)
- External system

Interface catalogue (provider -> consumer : key operations)

IMemberAPI (API Gateway -> Member Portal): searchBooks, placeHold, renewLoan, viewFines
ILibrarianAPI (API Gateway -> Librarian Console): addTitle, updateTitle, deactivateTitle, checkInBook
ICatalogAPI (Catalog -> API Gateway): searchByIsbn(query, page) - paginated, < 150 ms (NFR-001)
IHoldAPI (Reservation -> API Gateway): createHold(isbn, memberId), cancelHold, getQueuePosition
ICirculationAPI (Circulation -> API Gateway): checkOut, checkIn, renew(loanId)
IFineAPI (Fine -> API Gateway): getFines(memberId), settleFine
ICatalogLookup (Catalog -> Reservation): getTitle(isbn), getCopyCount(isbn)
IHoldQueue (Reservation -> Circulation): promoteNextHold(isbn), hasActiveHold(isbn)
IHoldNotify / ILoanNotify (Notification -> Reservation / Circulation): sendHoldReady, sendHoldLapsed, sendReceipt
IFinePosting (Fine -> Circulation): postOverdueFine(loanId, daysOverdue)
ICatalogData / IHoldData / ILoanData / IFineData (each DB -> its owning service): CRUD over SQL / JDBC
ISendMessage (Email / SMS Provider -> Notification): sendEmail(to, body), sendSms(to, body)